

Jadavpur University

UML LAB ASSIGNMENT REPORT

LAB Assignment Report Part-A PG/ITE/PC/T/122

Submitted by

NAME OF THE CANDIDATE: SUMIT RAJAK

ROLL NO.: 002511002016

BRANCH: IT

Year: 2025-26



JADAVPUR UNIVERSITY DEPARTMENT: INFORMATION TECHNOLOGY

Salt Lake Bypass, LB Block, Sector 3, Bidhannagar, Kolkata

Design and Implementation of an Object-Oriented Project Management System with UML Modelling and a Web-Based Interface

Sumit Rajak

Roll No. 002511002016, Department of Information Technology
Jadavpur University, Kolkata, India

Abstract—This report presents the design and implementation of an object-oriented Project Management System (PMS) developed to model how a software company manages multiple clients, projects, and employees. The system is designed around an abstract Project superclass with three polymorphic subclasses, and implements five core features specified in the assignment: employee-to-project assignment, requirement tracking, a four-stage development life-cycle tracker, project billing, and stage-wise client feedback. A Unified Modeling Language (UML) class diagram was produced to formally represent the system architecture prior to implementation. The system was implemented in Python and validated using an automated suite of fourteen unit tests, all of which pass. In addition to the core console-based system, a full-stack web interface was built using Flask and vanilla HTML/CSS/JavaScript, exposing the system through a browser-based dashboard, and deployed to a public hosting platform. The complete source code, UML diagrams, test results, and console/web output are documented herein and made available through a public GitHub repository.

Index Terms—Object-Oriented Programming, UML Class Diagram, Project Management System, Software Design, Polymorphism, Flask, Web Application, Software Testing.

I. Introduction

Modern software companies routinely manage a portfolio of concurrent client engagements, each requiring a dedicated subset of employees, a tracked development life-cycle, and independent billing and feedback records. This assignment requires the design of an object-oriented system that models this scenario, the production of a UML class diagram representing that design, and a working implementation demonstrating the system end-to-end.

The assignment brief (PG/ITE/PC/T/122, issued 24 June 2026, due 10 July 2026) specifies Project as the mandatory superclass, requires student-defined subclasses, and enumerates five features to be supported: (i) employee-to-project assignment recording designation and skill, (ii) free-text project requirements, (iii) a four-stage development tracker — Requirement Feasibility, Testing, Deployment, and Maintenance — (iv) project billing, and (v) client feedback captured at each life-cycle stage. A frontend/web interface is listed as an optional enhancement;

TABLE I. FEATURE-TO-CLASS TRACEABILITY

Assignment Feature	Responsible Class(es)
Employee – project assignment	Assignment, Employee, Project
Project requirements	Requirement
Development life-cycle tracker	DevelopmentTracker
Project billing	Billing
Client feedback per stage	Feedback

IV. System Design and UML Modelling

A. Class Responsibilities

Company is the aggregate root, holding references to all Employee, Client, and Project objects. Client owns the Projects it commissions. Project is the abstract superclass; WebDevelopmentProject, MobileAppProject, and

this report treats it as a full requirement and documents a complete deployed web application.

II. Objective

The objectives of this assignment are to: (1) apply core object-oriented principles — abstraction, encapsulation, inheritance, polymorphism, association, aggregation, and composition — to a realistic business scenario; (2) translate the resulting design into a formal UML class diagram; (3) implement a working system in Python satisfying all five specified features; (4) validate correctness through automated unit testing; and (5) extend the console-based system with a browser-accessible web interface, deployed publicly.

III. System Analysis

Table I maps each feature specified in the assignment to the class(es) responsible for implementing it, establishing direct traceability between the requirements and the design presented in Section IV.

Assignment Feature	Responsible Class(es)
Company / client management	Company, Client

The Project class is declared abstract and cannot be instantiated directly; every concrete project is one of three subclasses described in Section IV, each overriding two abstract methods, `calculate_billing()` and `get_deliverables()`, to demonstrate polymorphism.

DataScienceProject are the three concrete subclasses used to satisfy the assignment's instruction to “make your own sub-classes accordingly.” Each Project composes exactly one DevelopmentTracker and one Billing object — these cannot exist independently of their owning project — and aggregates lists of Requirement, Assignment, and Feedback objects.

B. OOP Principles Applied

Table II summarises where each core object-oriented principle is realised in the implementation.

TABLE II. OBJECT-ORIENTED PRINCIPLES AND THEIR REALISATION

Principle	Realisation in the System
Abstraction	Project is declared as an abstract base class (ABC)
Encapsulation	All attributes are private; accessed via @property getters
Inheritance	Three subclasses extend Project
Polymorphism	calculate_billing() behaves differently per subclass
Composition	Project owns DevelopmentTracker and Billing

Principle	Realisation in the System
Aggregation	Company aggregates Employee, Client, Project
Association	Employee ↔ Project linked via Assignment

Fig. 1 presents the complete UML class diagram generated directly from the implemented class structure, showing all attributes, methods, and relationship types (inheritance, composition, and aggregation) with multiplicities.

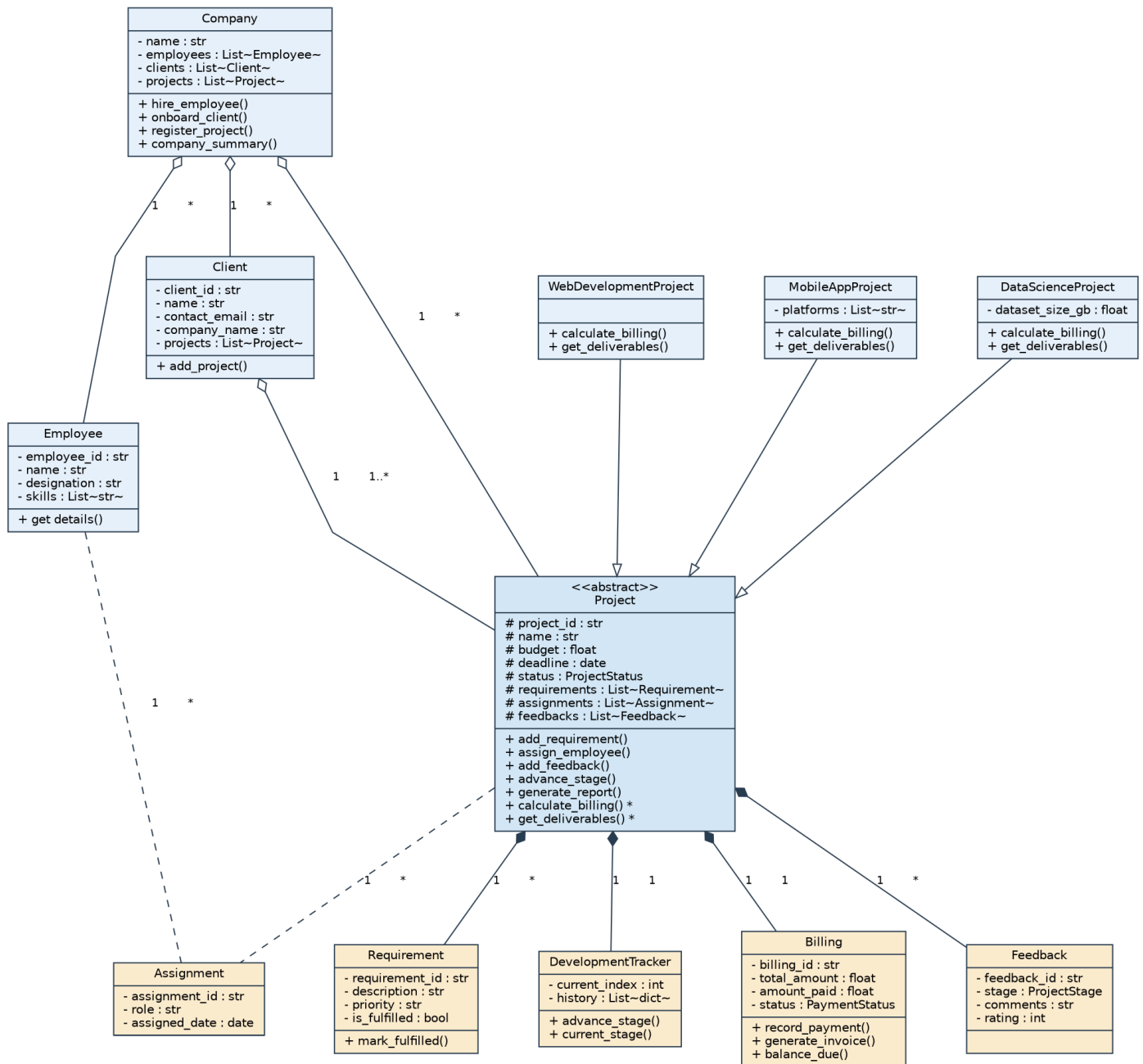


Fig. 1. UML class diagram of the Project Management System, showing inheritance (hollow triangle), composition (filled diamond), and aggregation (hollow diamond) relationships.

V. Implementation

The system is implemented in Python 3. The Project superclass is declared using the abc module so that it cannot be instantiated directly and so that subclasses are contractually required to implement `calculate_billing()` and `get_deliverables()`:

```
from abc import ABC, abstractmethod

class Project(ABC):
    @abstractmethod
    def calculate_billing(self): ...

    @abstractmethod
    def get_deliverables(self): ...
```

Each subclass overrides these methods with distinct pricing logic: `WebDevelopmentProject` applies a flat rate; `MobileAppProject` adds a 10% surcharge per additional target platform beyond the first; and `DataScienceProject` applies a fixed compute surcharge when the client's dataset exceeds 50

TABLE III. UNIT TEST COVERAGE SUMMARY

Test Class	Cases	Result
TestEmployeeAndClient	2	Pass
TestProjectLifecycle	4	Pass
TestAssignmentAndFeedback	3	Pass
TestBilling	4	Pass

GB. This is confirmed empirically in Section VIII using the actual billing figures produced by the running system.

The `DevelopmentTracker` enforces the exact four-stage ordering required by the assignment — Requirement Feasibility, Testing, Deployment, Maintenance — and cannot skip stages; calling `advance_stage()` moves a project exactly one step forward, and the project's status automatically transitions to “Completed” on reaching Maintenance.

VI. Testing

Correctness was verified using Python's unittest framework across five test classes covering fourteen individual test cases: employee/client creation, life-cycle progression and boundary conditions, employee assignment and feedback (including rating clamping to the 1–5 range), billing computation for all three project subclasses, payment-status transitions, and company-level registration. Table III summarises the test coverage; Fig. 3 shows the full test run captured on the author's own machine.

Test Class	Cases	Result
TestCompany	1	Pass

All fourteen tests complete in under 0.01 s on the reference machine, confirming that the life-cycle ordering, billing formulas, and data relationships behave as designed.

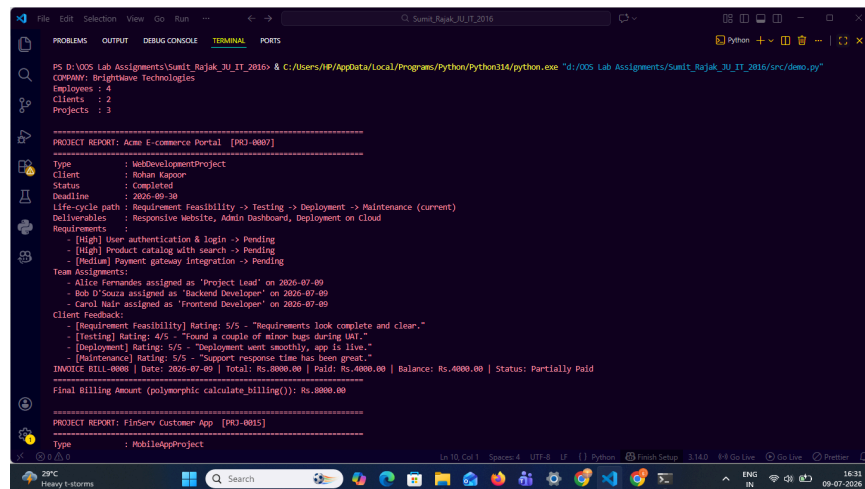


Fig. 2(a). Console output of `demo.py` — company summary and the Acme E-commerce Portal (`WebDevelopmentProject`) report.

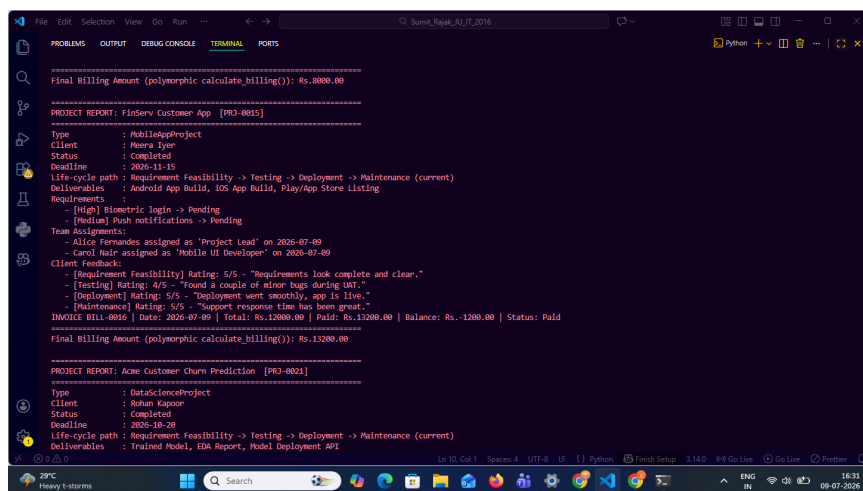


Fig. 2(b). Continued console output — FinServ Customer App (MobileAppProject) report showing the polymorphic platform surcharge.

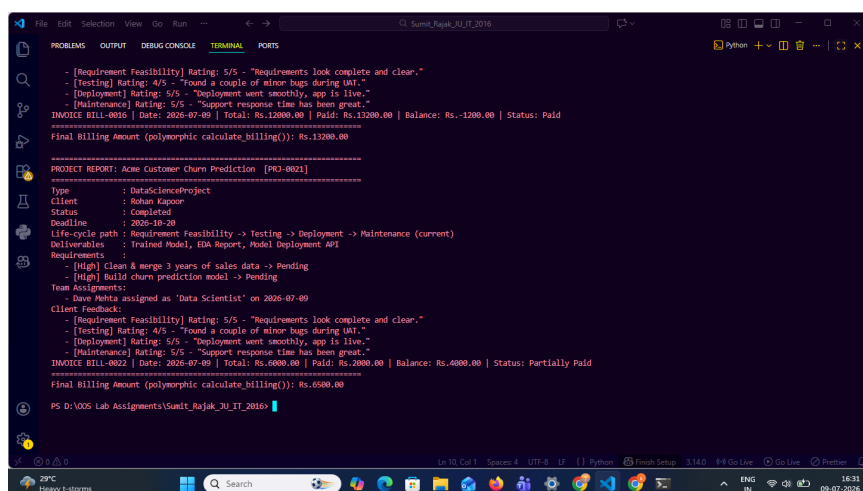


Fig. 2(c). Continued console output — Acme Customer Churn Prediction (DataScienceProject) report showing the dataset-size billing surcharge.

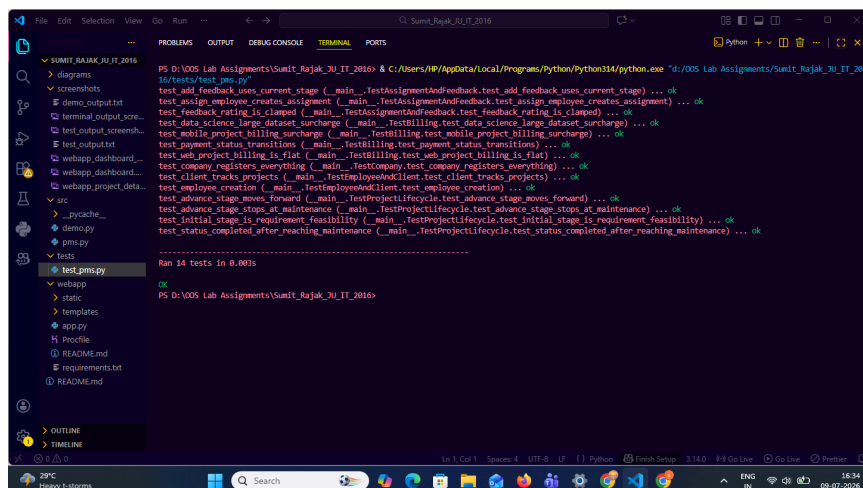


Fig. 3. Project file structure (VS Code Explorer) and the automated test run confirming all 14 unit tests pass in 0.003 s.

VII. Web Application Interface

To satisfy the assignment's optional frontend requirement as a complete deliverable rather than a minimal add-on, a full browser-based interface was built on top of the existing class

design. A Flask backend (app.py) exposes the system through a REST API; a single-page frontend (HTML, CSS, and vanilla JavaScript) consumes that API to provide a dashboard, project list, project detail view, and management forms for employees and clients. No part of the original class design in Section IV was modified — the web application is an additional interface onto the same objects exercised in Sections V and VI.

A. Architecture

A browser action (e.g. clicking “Advance stage”) triggers a JavaScript fetch() call to a Flask route (e.g. POST /api/projects/<id>/advance), which invokes the corresponding method on the in-memory Project object (project.advance_stage()) and returns the updated state as

JSON, which JavaScript uses to redraw the page without a reload.

B. Signature Interface Element

Each project's detail page includes a horizontal life-cycle rail — a stepper widget that visually renders the DevelopmentTracker's four stages and highlights the project's current position, directly corresponding to the assignment's life-cycle tracker requirement.

Fig. 4 shows the dashboard of the deployed application as captured on the author's machine, running under the customised company name “SUMIT BrightWave”.

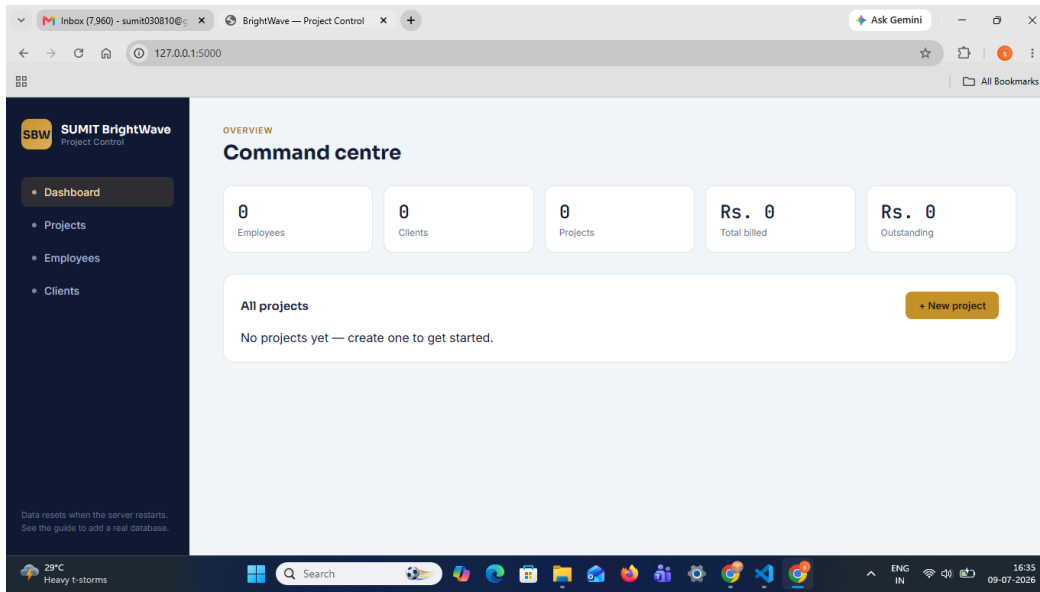


Fig. 4. Dashboard of the deployed web interface, running locally at 127.0.0.1:5000 prior to data entry.

VIII. Results and Discussion

The demonstration run in Fig. 2 registers one company (BrightWave Technologies) with four employees and two clients, and creates one project of each subclass. Table IV lists

TABLE IV. OBSERVED POLYMORPHIC BILLING RESULTS (FIG. 2)

Project (Subclass)	Budget	Final Billing
Acme E-commerce Portal (Web)	Rs. 8000	Rs. 8000
FinServ Customer App (Mobile, 2 platforms)	Rs. 12000	Rs. 13200
Acme Churn Prediction (Data Science, >50GB)	Rs. 6000	Rs. 6500

As expected, the WebDevelopmentProject instance bills at the flat budget rate, the MobileAppProject instance bills 10% above budget for its second target platform (Android and iOS), and the DataScienceProject instance bills a Rs. 500 surcharge for exceeding the 50 GB dataset threshold — each figure obtained directly from the actual console run rather than by manual calculation, confirming the implementation matches the design.

the final billing amounts actually produced by calculate_billing() for each project, confirming that the polymorphic pricing logic described in Section V behaves correctly on live data.

IX. Deployment and Source Code Availability

The complete source code — the core OOP system, the automated test suite, the UML diagram source, and the Flask/HTML/CSS/JS web application — is published in a public GitHub repository:

<https://github.com/SUMIT030810/project-management-system-uml>

The web application has additionally been deployed to Render and is publicly reachable at:

<https://project-management-system-uml.onrender.com/>

As the free hosting tier stores application state in memory, data entered on the deployed instance is cleared whenever the server restarts after a period of inactivity; this is a known limitation of the free tier and does not affect the correctness of

the underlying system, which is fully demonstrated locally in Fig. 2–4.

X. Conclusion

This report has presented the object-oriented design, UML modelling, implementation, and automated testing of a Project Management System satisfying all five features specified in assignment PG/ITE/PC/T/122, together with a fully working,

tested, and deployed web interface exceeding the assignment's optional frontend requirement. All fourteen unit tests pass, the UML diagram traces directly to the implemented classes, and the console and web outputs in Fig. 2–4 confirm the system behaves as designed. Future work could add persistent storage (e.g. SQLite) so that data entered through the web interface survives server restarts, and user authentication for multi-user deployments.

REFERENCES

- [1] G. Booch, J. Rumbaugh, and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Boston, MA: Addison-Wesley, 2005.
- [2] Object Management Group, “OMG Unified Modeling Language (UML) Specification,” [Online]. Available: <https://www.omg.org/spec/UML>. [Accessed: 9-Jul-2026].
- [3] Python Software Foundation, “Python 3 Language Reference,” [Online]. Available: <https://docs.python.org/3/>. [Accessed: 9-Jul-2026].
- [4] Pallets Projects, “Flask Documentation,” [Online]. Available: <https://flask.palletsprojects.com/>. [Accessed: 9-Jul-2026].
- [5] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA: Addison-Wesley, 1994.
- [6] S. Rajak, “project-management-system-uml,” GitHub repository, 2026. [Online]. Available: <https://github.com/SUMIT030810/project-management-system-uml>.
- [7] S. Rajak, “project-management-system-uml,” Web Interface, 2026. [Online]. Available: <https://project-management-system-uml.onrender.com/>